

Laborator 3 Arhitecturi Hardware Reconfigurabile

Proiectarea unor structuri hardware reconfigurabile dinamic. Circuitul PLA reconfigurabil

Conținut laborator:

1. Obiectivul lucrării.
2. Noțiuni teoretice
3. Cerințe laborator.
4. Temă.

1. Obiectivul lucrării:

În acest laborator se va prezenta unul din cele mai simple circuite reconfigurabile dinamic și anume automatul PLA reconfigurabil. Ca arhitectură, acesta este o matrice logică programabilă cu trei straturi de porți logice: un strat inversor, unul de porți SI și ultimul de porți SAU. În plus, interconexiunile dintre straturi se realizează prin intermediul unor multiplexoare. Intrările de selecție ale acestora stabilesc dacă o interconexiune este sau nu realizată. În total se obține un șir de intrări de selecție care constituie harta de configurații a circuitului. Exemplele din acest laborator sunt numai de circuite combinaționale. Totuși acest gen de circuit se poate utiliza și la implementarea de logică secvențială dacă se adaugă bistabili.

Utilitatea laboratorului: Circuitul PLA este primul circuit din seria celor reconfigurabile dinamic. Ca structură internă este cel mai simplu dar poate fi utilizat în numeroase aplicații în care se cere implementarea unor circuite de control a căror funcție de răspuns să fie variabilă în timp. Avantajul acestuia este rapiditatea în (re) configurare. Dezavantajul este creșterea dimensiunii cu numărul de intrări și ieșiri.

2. Noțiuni teoretice

Circuitul reconfigurabil min(max)termeni - $\mathfrak{R}$

Prima versiune de circuit reconfigurabil de tip PLA este mai apropiată de conceptul de sinteză „clasică”, care reduce totul la obținerea unor mintermeni sau maxtermeni.

Structura este prezentată în figura 1. Așa cum este ilustrat în figură, funcțiile elementare $e_j(x_1, x_2, \dots, x_{n_j})$ ale circuitului reconfigurabil sunt cunoscute și întotdeauna aceleași, astfel încât să genereze o sumă de produse logice. Acestea reprezintă funcțiile elementare. Sarcina algoritmului genetic este aceea de a determina ruta optimă (setul de conexiuni optime) pentru legătura între intrări (directe și negate) și stratul de porți SI. În figura 1 este prezentată varianta mintermeni, dar se poate ușor modifica și pentru varianta maxtermeni cu aceleași cerințe. Pentru exemplificare și reprezentare, în figură s-a folosit o versiune simplificată de circuit logic care permite implementarea oricărei funcții cu trei intrări și o ieșire.

Avantaje

Această versiune de circuit reconfigurabil dinamic este practic cea mai simplă posibilă. Algoritmul de reconfigurare utilizat are ca scop determinarea combinației optime pentru obținerea funcției obiectiv, prin configurarea stării conexiunii pentru fiecare poartă SI la intrare. Convergența este cea mai rapidă, iar soluțiile sunt cele mai apropiate de cele obținute prin metodele de sinteză „clasice”. Adâncimea circuitului este dată de adâncimea modulelor de „reconfigurare conexiuni” la care se adună 2 (rețeaua SI și rețeaua SAU).

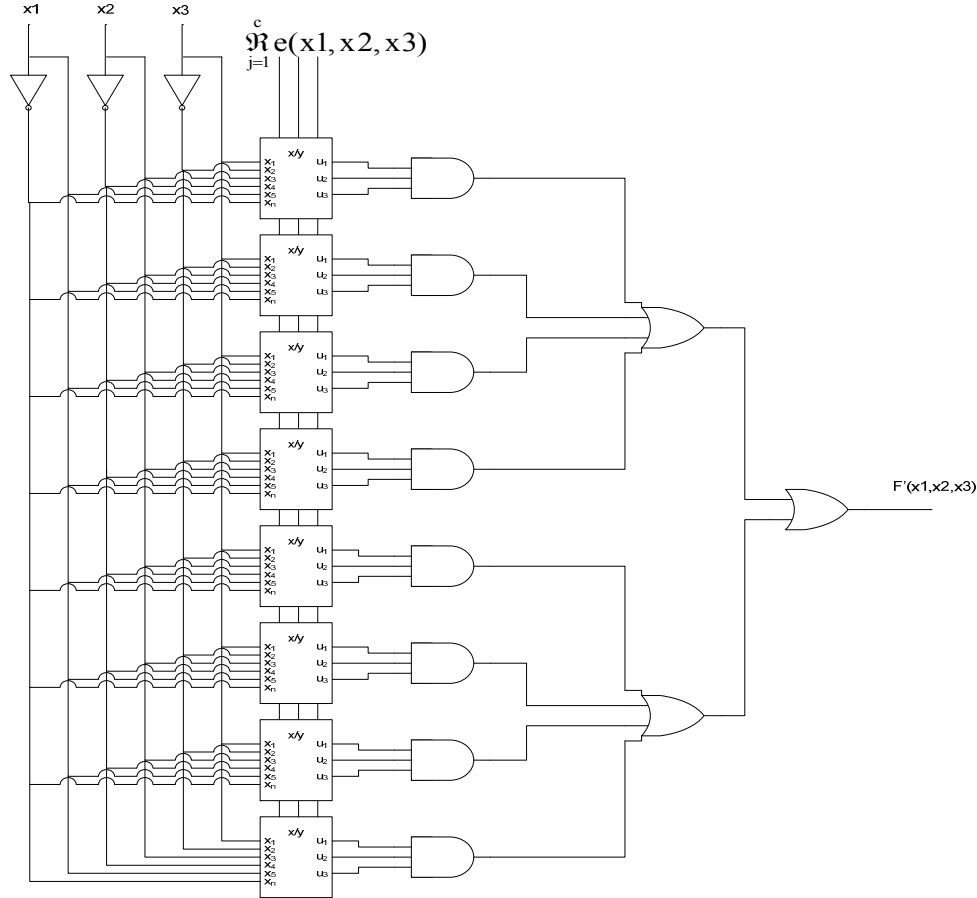


Fig.1. Circuitul reconfigurabil mintermeni - $\Re$ pentru o funcție binară $F(x_1, x_2, x_3)=y$

Dezavantaje

După cum am arătat, exemplul din figura 1 este pentru funcții logice cu 3 intrări și 1 ieșire. În cazul în care se dorește implementarea unei funcții logice cu 2 ieșiri dimensiunea circuitului se va dubla, deci și lungimea unui individ care codifică stările conexiunilor la un moment dat.

Un grup de tipul celui prezentat în figură, va avea pentru un număr de intrări egal cu i și număr de ieșiri egal cu j următoarele caracteristici:

Tabelul 1. Dimensiunea circuitului reconfigurabil mintermeni - $\mathfrak{R}$ pentru i intrări și j ieșiri

Parametru	Dimensiune
Număr circuite configurare conexiuni	$2^i \times j$
Intrări circuite configurare conexiuni	$2 \times i$
Ieșiri circuite configurare conexiuni	I
Porți strat SI	$2^i \times j$
Intrări porți strat SI	I
Porți strat SAU	J
Intrări porți strat SAU	2^i

Cu alte cuvinte, o creștere a numărului de intrări – ieșiri conduce la o creștere exponențială a dimensiunilor circuitului, dar adâncimea rămâne în principiu aceeași (bineînțeles de la un anumit număr de intrări pe o poartă tehnologic devine imposibil de implementat și se lucrează pe mai multe straturi așa cum s-a procedat cu poarta SAU din figura 1).

Circuitul ca model reconfigurabil dinamic poate fi privit ca o rețea reconfigurabilă dinamic unidimensională. Celulele de procesare sunt date aici de porțile logice, iar algoritmi de rutare sunt determinați de evoluția indivizilor.

Optimizare

Problema care se ridică la acest gen de circuit este următoarea: cum se poate optimiza structura pentru reducerea numărului de circuite? Răspunsul la întrebare se poate da părăsind complet orice analogie cu metodele „clasice” de sinteză și trecând pe un domeniu în care se vor utiliza algoritmi de reconfigurare de tipul celor de căutare și optimizare.

În primul rând, problema optimizării se pune în situația în care se dorește implementarea unor funcții logice cu mai mult de o ieșire. Pentru funcții logice cu numai o ieșire, așa cum se va arăta în capitolul următor, soluția de mai sus rămâne cea mai eficientă.

Dacă

$$F(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_m) \text{ cu } x_i \in \{0,1\}, y_j \in \{0,1\}$$

atunci putem avea un set de m funcții, de forma următoare:

$$F_1(x_1, x_2, \dots, x_n) = y_1, F_2(x_1, x_2, \dots, x_n) = y_2, \dots, F_m(x_1, x_2, \dots, x_n) = y_m.$$

Fiecare funcție cu o ieșire se poate reprezenta la rândul său folosind funcții elementare, cu o lege de compoziție așa cum a fost explicat mai sus:

$$F_1 = f_{11} \circ f_{12} \dots, F_2 = f_{21} \circ f_{22} \dots, \dots, F_m = f_{m1} \circ f_{m2} \dots$$

Se observă faptul că în situația în care:

$$f_{xi} \equiv f_{yj}, x \neq y$$

atunci apar funcții identice care se pot refolosi eficientizând și optimizând structura rezultată.

Circuitul reconfigurabil INV-AND-OR- $\mathfrak{R}$

Problema determinării funcțiilor sau grupurilor de funcții elementare care compun funcțiile binare cu o singură ieșire este una abordabilă prin . Pentru aceasta, circuitul reconfigurabil min(max)termeni - $\mathfrak{R}$ trebuie schimbat. Alături de conexiunile configurabile aflate între stratul de intrare (format din intrări și negația lor) și stratul de porți AND (SI), sunt introduse conexiuni reconfigurabile și între stratul de porți AND (SI) și stratul de porți OR (SAU) aflate la ieșire. În figura de mai jos este prezentat un exemplu de circuit reconfigurabil INV-AND-OR- $\mathfrak{R}$ care are 3 intrări și două ieșiri. Toate exemplele care sunt date aici sunt simplificate (număr de intrări și ieșiri relativ mici), dar ele pot fi extinse pentru obținerea unor rezultate semnificative.

Și la acest circuit funcțiile elementare sunt stabilite, având circuite elementare fixate. Spre deosebire de soluția mintermeni - $\mathfrak{R}$, aici avem un număr mai mare de configurări care se pot aplica circuitului pentru cele două straturi. Viteza de convergență este mai mică decât la versiunea anterioară, deoarece alături de determinarea termenilor de intrare în stratul SI sunt determinați și termenii de intrare în stratul SAU. Modelul este tot al unei rețele reconfigurabile dinamice unidimensionale deoarece putem vedea straturile de „circuite configurare conexiuni” ca fiind așezate într-o singură linie.

Avantaje

Soluția prezentată este mult mai eficientă din punct de vedere al spațiului ocupat față de cea anterioară. Adâncimea circuitului este dată de dimensiunea circuitelor de configurare pentru cele două straturi plus 2. Pentru un circuit general cu i intrări și j ieșiri avem următoarele:

Tabelul 2. Dimensiunea circuitului reconfigurabil INV-AND-OR- $\mathfrak{R}$

Parametru	Dimensiune
Număr circuite configurare conexiuni	$2^i + j$
Intrări circuite configurare conexiuni 1	$2 \times i$
Ieșiri circuite configurare conexiuni 1	I
Intrări circuite configurare conexiuni 2	2^i
Ieșiri circuite configurare conexiuni 2	2^i
Porți strat SI	2^i
Intrări porți strat SI	I
Porți strat SAU	J
Intrări porți strat SAU	2^i

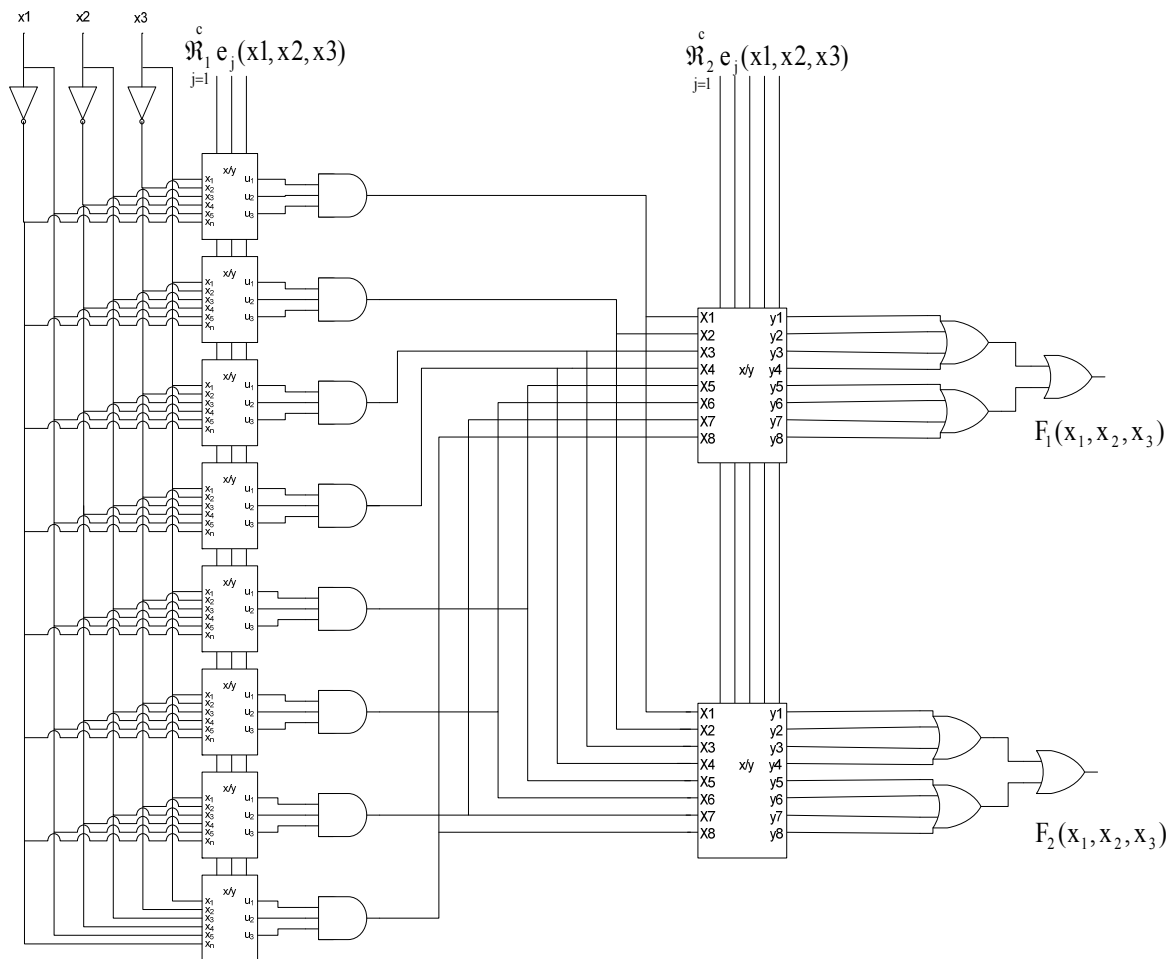


Fig.2. Circuitul reconfigurabil INV-AND-OR- $\mathfrak{R}$

Creșterea numărului de intrări conduce la creșterea exponențială a numărului de circuite de configurare din primul strat și al numărului de porți SI. În schimb, creșterea numărului de ieșiri duce la o creștere liniară a numărului de porți logice SAU și al circuitelor logice de configurare în stratul SAU. Toate acestea, în condițiile în care adâncimea circuitului rămâne la fel. Deși dimensiunea hărții de configurare (numărul de intrări de selecție din multiplexoare) crește, luând în considerare și numărul de circuite de conectare din cel de-al doilea strat, totuși, în cazul în care numărul de ieșiri este mare, se ajunge la valori mai mici decât la varianta anterioară.

Deci, deși a apărut un strat suplimentar de circuite de configurare (între stratul de porți SI și stratul de porți SAU), dimensiunea circuitului variază liniar cu numărul de ieșiri. În această situație, așa cum se va arăta în capitolul următor, pentru un anumit număr de ieșiri dimensiunea circuitului este semnificativ mai mică față de varianta anterioară.

Dezavantaje

Un prim dezavantaj pe care l-am menționat consta în creșterea complexității circuitului pentru un număr mic de ieșiri. Efectul apare în creșterea dimensiunii hărții de configurare, fapt care își va pune amprenta în creșterea timpului de convergență al algoritmului de reconfigurare. În al doilea rând, deși problematica sintezei s-a depărtat de soluțiile clasice prin determinarea mintermenilor, dar și prin determinarea funcțiilor elementare componente comune, ea rămâne tributară modului de proiectare clasic prin descrierea unor circuite ca sume de mintermeni sau produse de max termeni. Algoritmii moderni de reconfigurare sunt priviți ca o metodă de exploatare a spațiilor de proiectare inaccesibile prin metodele convenționale de proiectare și care pot aduce rezultate superioare. Însă pentru a permite acest lucru, trebuie îndepărtate orice restricții rezidente din conceptul de sinteză clasic ceea ce duce la exploatarea algoritmului genetic la întreaga lui putere de algoritm de căutare în spațiul de soluții a circuitelor. Pentru asta se pune problema generării unor circuite care nu au aceeași adâncime și nu au aceleași celule elementare.

Optimizare

Toate valorile specificate în tabelul 2 sunt date în condițiile în care se dorește descrierea circuitelor folosind funcții elementare utilizate și în proiectarea clasică a circuitelor, dar pot fi totuși modificate. De exemplu, se poate concepe un circuit care să nu aibă numai 3 straturi, ci să aibă 4 straturi de porți. Mărirea numărului de straturi ar reprezenta bineînțeles o supraevaluare a circuitului, dar aceasta se poate corecta prin introducerea unui număr mai mic de intrări pentru fiecare poartă din strat. Cu alte cuvinte se poate decide la o creștere în adâncime și la o scădere în lățime a circuitelor. Din acest punct de vedere se poate vorbi despre un număr de straturi, precum și despre un număr de circuite pe strat ca un set de parametri ai circuitului reconfigurabil, independent de numărul de intrări sau ieșiri.

Alături de calculul regulilor de routare, optimizarea înseamnă și configurarea funcțiilor logice elementare. Aceasta presupune însă proiectarea unor celule reconfigurabile care să permită aspectele precizate anterior. În al doilea rând, se pune problema generării unor circuite cu adâncimi diferite, fapt care implică și refacerea modului de routare. La cele două modele de circuite reconfigurabile, sensul de routare era întotdeauna de la intrare la ieșire trecând prin două straturi de porți – niciodată nu se putea evita trecerea printr-un strat sau altul. De aceea, toate circuitele generate cu primele două metode au adâncimi egale, fără să se ridice problema unei optimizări a circuitului generat în această direcție.

Cerințe laborator:

Prezentare montaj experimental:

În acest laborator se va construi o structură hardware reconfigurabilă de tipul PLA care permite atât sinteza circuitelor cu min termeni cât și cu max termeni. Funcția țintă va fi de forma $F(x_1, x_2, x_3) = y$ adică orice funcție logică ce are 3 intrări și o ieșire.

Schema circuitului propus spre a fi realizat în laborator este prezentată în figura de mai jos:

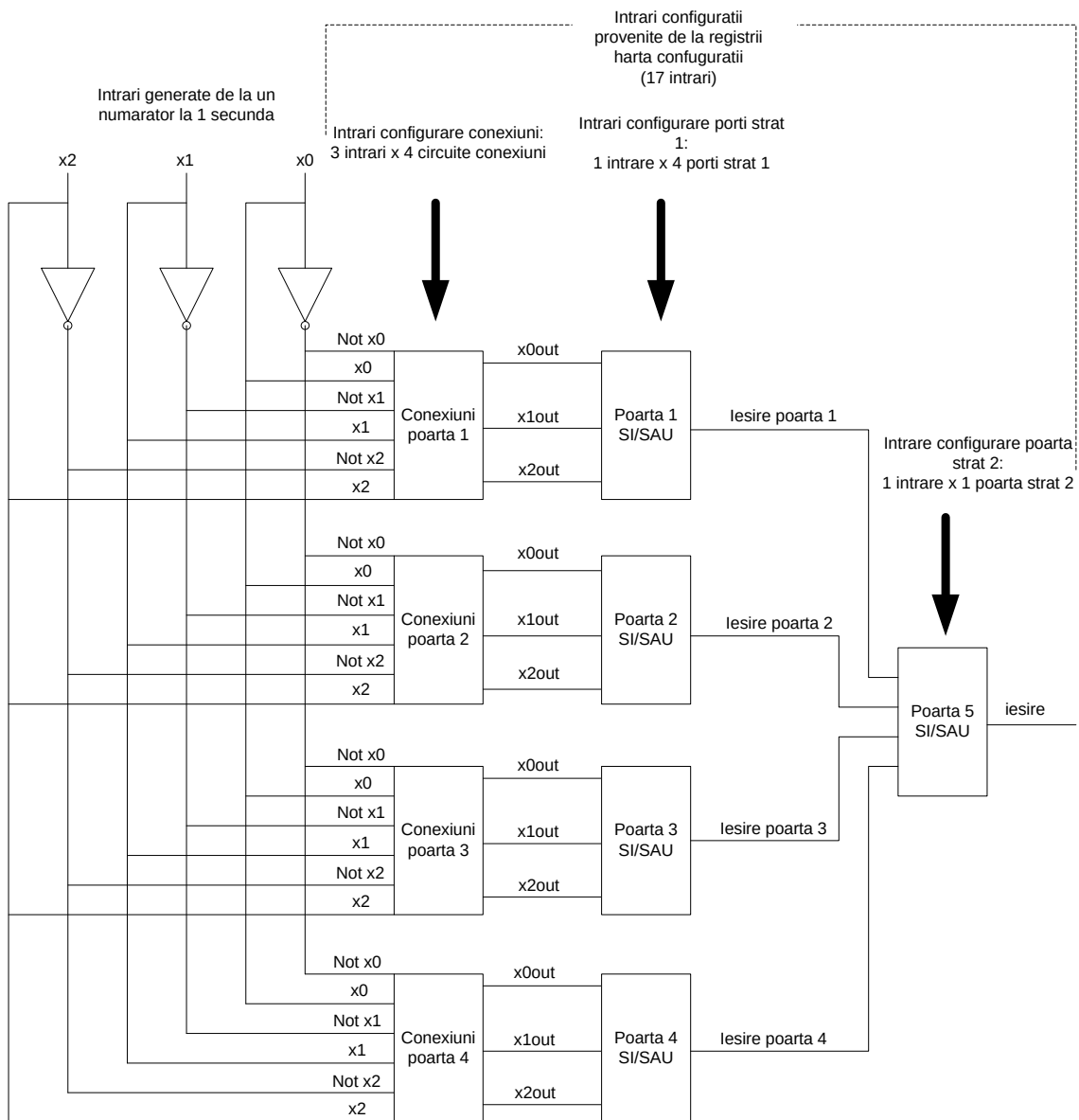


Fig.3. Schema circuit reconfigurabil mintermeni - maxtermeni - $\mathfrak{N}$ pentru o funcție binară $F(x_1, x_2, x_3) = y$

Schema o urmărește pe cea a circuitului reconfigurabil mintermeni - $\mathfrak{N}$ prezentată în figura 1, cu următoarele îmbunătățiri. În loc de porți logice fixe s-au adăugat porți logice configurabile care permit selecția funcției SI sau a funcției SAU. Astfel, stratul de porți SI și stratul de porți SAU pot fi acum interschimbate ceea ce permite implementarea atât a descrierilor utilizând mintermeni cât și a descrierilor care utilizează maxtermeni.

Din acest motiv, schema a fost îmbunătățită în sensul reducerii numărului de porți din stratul 1 la jumătate.

Montajul experimental care se va implementa pe macheta de laborator va avea structura prezentată în figura 4.

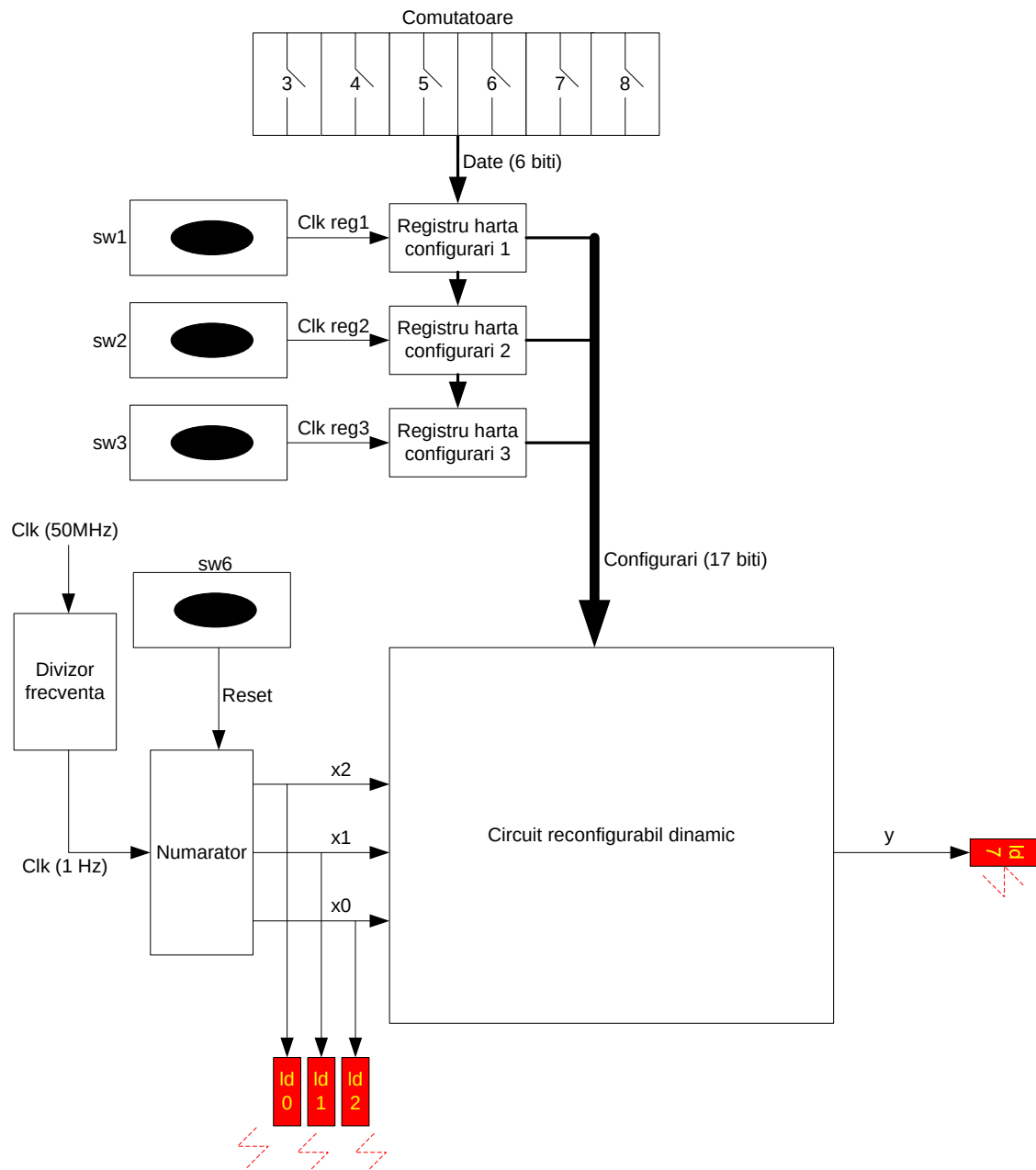


Fig.4. Montaj experimental pentru circuitul reconfigurabil

Intrările din circuitul reconfigurabil sunt generate cu ajutorul unui numărător pe 3 biți cu frecvența de 1 Hz. Astfel, vor fi parcurse toate stările pe intrare și se vor vedea toate valorile ieșirii. Cele 17 intrări de configurare care sunt necesare pentru circuitul reconfigurabil vor fi generate în grupe de câte 6 de la comutatoare și vor fi stocate în regiștri cu încărcare paralelă pe 6 biți fiecare.

Harta configurațiilor este dată în tabelele de mai jos:

Tabelul 3:

Registrul 1:

5	4	3	2	1	0
Conexiuni poarta 2 (0 – notX, 1 – X)			Conexiuni poarta 1 (0 – notX, 1 – X)		
X2out	X1out	X0out	X2out	X1out	X0out

Registrul 2:

5	4	3	2	1	0
Conexiuni poarta 4 (0 – notX, 1 – X)			Conexiuni poarta 3 (0 – notX, 1 – X)		
X2out	X1out	X0out	X2out	X1out	X0out

Registrul 3:

5	4	3	2	1	0
Setări porți logice (0 – SI, 1 – SAU)					
-	Poarta 5	Poarta 4	Poarta 3	Poarta 2	Poarta 1

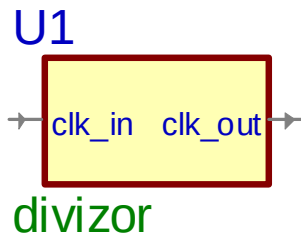
Configurațiile se introduc pe rând în cei trei regiștri, așa cum este prezentat mai sus, utilizând magistrala de date de intrare comună conectată la comutatoarele de pe machetă. Pentru înscrierea registrului se generează un semnal de ceas utilizând butoanele de pe machetă.

Construirea montajului experimental pe machetele cu FPGA utilizând VHDL

Se va crea un proiect în ActiveHDL (pentru detalii revedeți anexa laborator!). În proiect se vor crea următoarele fișiere VHDL (utilizând Wizard):

1. Divizor.vhd în care se va descrie divizorul de frecvență necesar pentru generarea semnalului de ceas cu frecvența de 1Hz

Caracteristicile IO acestui circuit sunt următoarele:



Codul VHDL este următorul:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity divizor is
    port(
        clk_in : in STD_LOGIC;
```

```

        clk_out : out STD_LOGIC
    );
end divizor;

architecture divizor_a of divizor is
begin

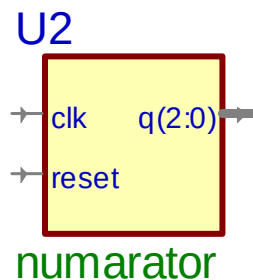
    process(clk_in)
        variable cnt_int : integer;
    begin
        if clk_in'event and clk_in='1' then
            if cnt_int < 500000000 then
                cnt_int := cnt_int + 1;
                clk_out <= '0';
            else
                cnt_int := 0;
                clk_out <= '1';
            end if;
        end if;
    end process;

end divizor_a;

```

2. Numărător.vhd în care se va descrie numărătorul pentru generarea intrărilor utilizat la evaluarea circuitului

Caracteristicile IO ale circuitului sunt prezentate în figura de mai jos:



Codul în VHDL este:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

entity numarator is
    port(
        clk : in STD_LOGIC;

```

```

        reset : in STD_LOGIC;
        q : out STD_LOGIC_VECTOR(2 downto 0)
    );
end numarator;

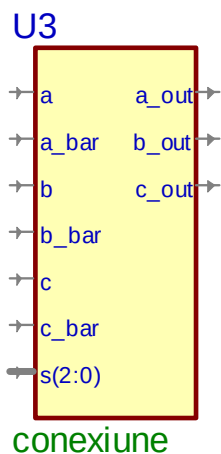
architecture numarator_a of numarator is
begin

    process(clk,reset)
        variable cnt_int : std_logic_vector(2 downto 0);
    begin
        if clk'event and clk='1' then
            if reset = '0' then
                cnt_int := "000";
            else
                cnt_int := cnt_int + '1';
            end if;
            q <= cnt_int;
        end if;
    end process;

end numarator_a;

```

3. Conexiune.vhd reprezintă descrierea circuitului responsabil cu stabilirea conexiunilor între stratul de porți inversoare și primul strat de porți configurabile. Circuitul este compus din 3 multiplexoare 2:1, așa cum se poate vedea în descrierea VHDL



Circuitul are 6 intrări, 3 selecții și 3 ieșiri. La fiecare ieșire (a_out, b_out și c_out) se poate conecta una din cele două intrări care conțin în denumire prima literă (de ex. la a_out se poate conecta a sau a_bar). Pentru intrările din grupul „a” este alocată intrarea de selecție s(0), pentru cele din grupul „b” intrarea de selecție s(1) și pentru cele din grupul „c” intrarea de selecție s(2).

Dacă intrarea de selecție corespunzătoare grupului este 0 logic atunci pe ieșirea omologă este conectată intrarea cu terminația _bar în timp ce dacă pe selecția corespunzătoare grupului se află 1 logic atunci se conectează intrarea fără terminația _bar din grupul respectiv la ieșirea grupului.

De exemplu: pe ieșirea a_out se conectează intrarea a dacă s(0) este 1 logic sau intrarea a_bar dacă s(0) este 0 logic.

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.all;
```

```
entity conexiune is
```

```
    port(
```

```
        a : in STD_LOGIC;
```

```
        a_bar : in STD_LOGIC;
```

```
        b : in STD_LOGIC;
```

```
        b_bar : in STD_LOGIC;
```

```
        c : in STD_LOGIC;
```

```
        c_bar : in STD_LOGIC;
```

```
        s : in std_logic_vector(2 downto 0);
```

```
        a_out : out STD_LOGIC;
```

```
        b_out : out STD_LOGIC;
```

```
        c_out : out STD_LOGIC
```

```
    );
```

```
end conexiune;
```

```
architecture conexiune_a of conexiune is
```

```
begin
```

```
    --multiplexorul care selecteaza prima intrare
```

```
    with s(0) select
```

```
    a_out <= a when '1',
```

```
    a_bar when others;
```

```
    --multiplexorul care selecteaza a doua intrare
```

```
    with s(1) select
```

```
    b_out <= b when '1',
```

```
    b_bar when others;
```

```
    --multiplexorul care selecteaza a treia intrare
```

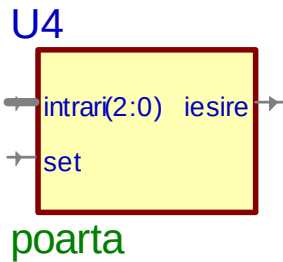
```
    with s(2) select
```

```
    c_out <= c when '1',
```

```
    c_bar when others;
```

```
end conexiune_a;
```

4. Poarta.vhd reprezintă poarta configurabilă din primul strat.



Aceasta este o funcție logică cu trei intrări și o ieșire. Funcția logică este de tipul SI dacă intrarea set este 0 logic și de tipul SAU dacă intrarea set este 1 logic.

Codul în VHDL pentru poarta configurabilă este prezentat mai jos. Așa cum se poate observa, este constituită din două porți logice: SI și SAU și dintr-un multiplexor 2:1.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity poarta is
    port(
        intrari : in STD_LOGIC_VECTOR(2 downto 0);
        set : in std_logic;
        iesire : out STD_LOGIC
    );
end poarta;

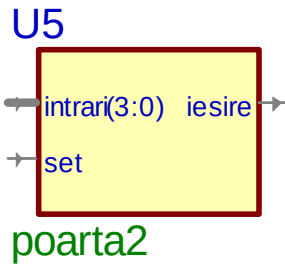
architecture poarta_a of poarta is
    signal si_out, sau_out : std_logic;
begin

    si_out <= intrari(0) and intrari(1) and intrari(2);
    sau_out <= intrari(0) or intrari(1) or intrari(2);

    with set select
        iesire <= sau_out when '1',
        si_out when others;

end poarta_a;
```

5. Poarta2.vhd reprezintă poarta configurabilă din cel de-al doilea strat.



Structura ei este similară cu cea din primul strat numai că funcțiile logice sunt cu 4 intrări și o ieșire.

Codul în VHDL este prezentat mai jos.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity poarta2 is
    port(
        set : in STD_LOGIC;
        intrari : in STD_LOGIC_VECTOR(3 downto 0);
        iesire : out STD_LOGIC
    );
end poarta2;

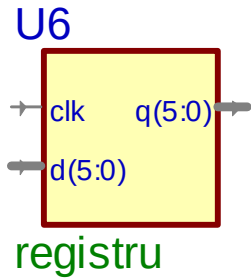
architecture poarta2_a of poarta2 is
    signal si_out, sau_out: std_logic;
begin

    si_out <= intrari(0) and intrari(1) and intrari(2) and intrari(3);
    sau_out <= intrari(0) or intrari(1) or intrari(2) or intrari(3);

    with set select
        iesire <= sau_out when '1',
        si_out when others;

end poarta2_a;
```

6. Registru.vhd este descrierea registrului cu acces paralel pe 6 biți utilizat pentru stocarea unei părți din harta de configurații.



Codul în VHDL pentru acesta este prezentat în continuare:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity registru is
    port(
        clk : in STD_LOGIC;
        d : in STD_LOGIC_VECTOR(5 downto 0);
        q : out STD_LOGIC_VECTOR(5 downto 0)
    );
end registru;

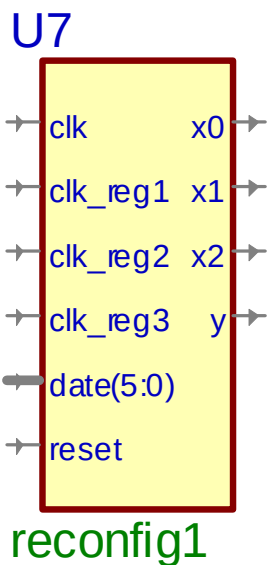
architecture registru_a of registru is
begin

    process(clk,d)
    begin
        if clk'event and clk='1' then
            q <= d;
        end if;
    end process;

end registru_a;
```

7. Reconfig1.vhd care conține circuitul final (montaj experimental)

Structura acestuia ca pini de intrare ieșire este prezentată mai jos.



Pinii alocați la machetă pentru intrările și ieșirile circuitului, în conformitate cu figura 4, vor fi următorii:

Tabelul 4: Alocarea pinilor

Pin logic circuit	Denumire pin machetă
Clk	Gclk
Clk_reg1	Sw_user0
Clk_reg2	Sw_user1
Clk_reg3	Sw_user2
Date(0)	Sw_dip7
Date(1)	Sw_dip6
Date(2)	Sw_dip5
Date(3)	Sw_dip4
Date(4)	Sw_dip3
Date(5)	Sw_dip2
Reset	Sw_user5
X0	Led2
X1	Led1
X2	Led0
Y	Led7

Pinii vor fi introduși în utilitarul de asociere constrângeri PACE consultând tabelul cu alocarea pinilor FPGA de la sfârșitul anexei.

Descrierea în VHDL este prezentată în continuare:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

```
entity reconfig1 is
```



```

port(
    clk : in STD_LOGIC;
    clk_reg1 : in STD_LOGIC;
    clk_reg2 : in STD_LOGIC;
    clk_reg3 : in STD_LOGIC;
    reset : in STD_LOGIC;
    date : in STD_LOGIC_VECTOR(5 downto 0);
    x0 : out STD_LOGIC;
    x1 : out STD_LOGIC;
    x2 : out STD_LOGIC;
    y : out STD_LOGIC
);
end reconfig1;

```

architecture reconfig1_a of reconfig1 is

--declaratii componente

component divizor is

```

port(
    clk_in : in STD_LOGIC;
    clk_out : out STD_LOGIC
);

```

end component;

component numarator is

```

port(
    clk : in STD_LOGIC;
    reset : in STD_LOGIC;
    q : out STD_LOGIC_VECTOR(2 downto 0)
);

```

end component;

component conexiune is

```

port(
    a : in STD_LOGIC;
    a_bar : in STD_LOGIC;
    b : in STD_LOGIC;
    b_bar : in STD_LOGIC;
    c : in STD_LOGIC;
    c_bar : in STD_LOGIC;
    s : in std_logic_vector(2 downto 0);
    a_out : out STD_LOGIC;
    b_out : out STD_LOGIC;
    c_out : out STD_LOGIC
);

```

end component;

component poarta is

```

port(
    intrari : in STD_LOGIC_VECTOR(2 downto 0);

```

```

        set : in std_logic;
        iesire : out STD_LOGIC
    );
end component;
component poarta2 is
    port(
        set : in STD_LOGIC;
        intrari : in STD_LOGIC_VECTOR(3 downto 0);
        iesire : out STD_LOGIC
    );
end component;
component registru is
    port(
        clk : in STD_LOGIC;
        d : in STD_LOGIC_VECTOR(5 downto 0);
        q : out STD_LOGIC_VECTOR(5 downto 0)
    );
end component;
--declaratii semnale
signal clk_num : std_logic;
signal intrari : std_logic_vector(2 downto 0);
signal not_intrari : std_logic_vector(2 downto 0);
signal intrari_strat1 : std_logic_vector(11 downto 0);
signal intrari_strat2 : std_logic_vector(3 downto 0);
signal harta_configurari : std_logic_vector(16 downto 0);
signal iesire_registrii : std_logic_vector(17 downto 0);

begin

    cddivizor : divizor port map(clk,clk_num);
    cnumerator : numerator port map(clk_num,reset,intrari);
    not_intrari <= not intrari;
    --primul strat conexiuni
    gconexiune: for i in 0 to 3 generate
        cconexiune:
            conexiune
            port
            map(intrari(0),not_intrari(0),intrari(1),not_intrari(1),intrari(2),not_intrari(2),harta_configurari(i*3+2 downto i*3),intrari_strat1(i*3),intrari_strat1(i*3+1),intrari_strat1(i*3+2));
        end generate;
    --primul strat porti
    gporti: for i in 0 to 3 generate
        cpoarta:
            poarta
            port
            map(intrari_strat1(i*3+2 downto i*3),harta_configurari(12+i),intrari_strat2(i));
        end generate;
    --al doilea strat de porti
    cpoarta2: poarta2 port map(harta_configurari(16),intrari_strat2,y);
    --pentru afisare intrari

```

```

x0 <= intrari(0);
x1 <= intrari(1);
x2 <= intrari(2);
--conectare harta configuratii la iesire registrilor
harta_configurari <= iesire_registrii(16 downto 0);

--registrii harta configuratii
cregistru1 : registru port map(clk_reg1,date,iesire_registrii(5 downto 0));
cregistru2 : registru port map(clk_reg2,date,iesire_registrii(11 downto 6));
cregistru3 : registru port map(clk_reg3,date,iesire_registrii(17 downto 12));

end reconfig1_a;

```

După realizarea descrierii se va implementa pe machetă urmărind pașii descriși în anexă. La implementarea pe machetă se va ține cont de faptul că toți regiștrii au inițial încărcată valoarea 0. Aceasta este echivalent cu faptul că în circuitul reconfigurabil se află la pornire implementată o funcție SI logic.

Pentru implementarea unei funcții SAU logic cu trei intrări trebuie scriși toți regiștrii de configurare cu valoarea 1 logic.

În continuare se vor implementa alte funcții logice pentru a ilustra puterea de reconfigurare a circuitului.

Probleme de rezolvat în laborator:

1. Pe baza descrierilor în VHDL să se deseneze schema circuitului Conexiune și a circuitului Poarta
2. Să se implementeze pe circuitul reconfigurabil dinamic o poartă XOR cu 3 intrări. Tabela de adevăr a acesteia este următoarea:

X2	X1	X0	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

3. Să se implementeze pe circuitul reconfigurabil dinamic un sumator cu 3 intrări. Tabela de adevăr a acesteia este următoarea:

X2	X1	X0	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0

1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

4. Să se implementeze pe circuitul reconfigurabil dinamic o funcție majoritară 1.
Tabelul de adevăr este:

X2	X1	X0	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Temă

1. Să se construiască un circuit reconfigurabil de tip PLA pentru funcții $F(x_0, x_1, x_2) = y$ cu porțile configurabile care pot fi SI, SAU, SI-NU, SAU-NU. Care este dimensiunea în biți a hărții de configurații?
2. Să se construiască un circuit reconfigurabil de tip PLA mintermeni - $\mathfrak{R}$ pentru funcții $F(x_0, x_1, x_2) = (y_0, y_1)$. Care este dimensiunea în biți a hărții de configurații.
3. Ce modificări trebuie aduse circuitului realizat în acest laborator pentru ca la un moment dat să fie posibilă comutarea de la o harta de configurări la alta prin apăsarea unui buton. Să se construiască schema.
4. Un robot autonom primește informații din mediu pe 5 intrări și generează comenzi pe o 3 ieșire digitale (în formatul totul sau nimic). Să se construiască un circuit reconfigurabil (de tipul PLA mintermeni - $\mathfrak{R}$ astfel încât de la momentul inițializării să efectueze operația de adunare . Din momentul în care a suma obținută a depășit valoarea 4 își va schimba funcția în majoritar 1, generând pe toate cele trei ieșiri răspunsul la funcția majoritar.

Criterii de evaluare:

Cerințe laborator	
1	2P
2	2P
3	3P
4	3P
Temă	
1	2P
2	3P
3	2P
4	3P